

Code:

```
#include <stdio.h> #include <string.h>

#define MAXN 505 #define MAXM 2005

int head[MAXN], to[MAXM], cap[MAXM], nxt[MAXM], edge_cnt = 1; int level[MAXN],
q[MAXN], ptr[MAXN]; int vis[MAXN];

struct OriginalEdge { int u, v; } orig[MAXM]; int m_count = 0;

void link(int i, int h) { to[++edge_cnt] = h; cap[edge_cnt] = 1; nxt[edge_cnt] = head[i];
head[i] = edge_cnt; to[++edge_cnt] = i; cap[edge_cnt] = 1; nxt[edge_cnt] = head[h];
head[h] = edge_cnt; }

int bfs(int n, int s, int t) { memset(level, -1, sizeof(int) * (n + 1)); int front = 0, back = 0;
q[back++] = s; level[s] = 0;

while (front < back) {
    int u = q[front++];
    for (int e = head[u]; e; e = nxt[e]) {
        int v = to[e];
        if (cap[e] > 0 && level[v] == -1) {
            level[v] = level[u] + 1;
            q[back++] = v;
        }
    }
}
return level[t] != -1;
}

int dfs_flow(int u, int t, int pushed) { if (pushed == 0 || u == t) return pushed; for (int *cid =
&ptr[u]; *cid; *cid = nxt[*cid]) { int e = *cid; int v = to[e]; if (level[u] + 1 != level[v] || cap[e]
== 0) continue; int tr = dfs_flow(v, t, pushed < cap[e] ? pushed : cap[e]); if (tr == 0)
continue; cap[e] -= tr; cap[e ^ 1] += tr; return tr; } return 0; }

void dfs_cut(int u) { vis[u] = 1; for (int e = head[u]; e; e = nxt[e]) { int v = to[e]; if (cap[e] > 0
&& !vis[v]) { dfs_cut(v); } } }

int main() { int n, m; if (scanf("%d %d", &n, &m) != 2) return 0;
```

```

for (int i = 0; i < m; ++i) {
    int a, b;
    scanf("%d %d", &a, &b);
    link(a, b);
    orig[m_count++] = (struct OriginalEdge){a, b};
}

while (bfs(n, 1, n)) {
    for (int i = 1; i <= n; ++i) ptr[i] = head[i];
    while (1) {
        int pushed = dfs_flow(1, n, 1e9);
        if (!pushed) break;
    }
}

dfs_cut(1);

int ans_count = 0;
for (int i = 0; i < m_count; ++i) {
    int u = orig[i].u;
    int v = orig[i].v;
    if (vis[u] != vis[v]) {
        ans_count++;
    }
}

printf("%d\n", ans_count);
for (int i = 0; i < m_count; ++i) {
    int u = orig[i].u;
    int v = orig[i].v;
    if (vis[u] != vis[v]) {
        printf("%d %d\n", u, v);
    }
}

return 0;

}

```

Input:

```
2
3 4
1 4
```

Output:

```
2
4
1 3 5 6
3
1 2 6
```